Page: Details | Result: 6 - 724 - spmv_warp | Add Baseline | Apply Rules | Occupancy Calculator | Source Comparison | Save as PDF

	Result	Time	Cycles	GPU	SM Frequency	Process	Attributes
Current	724 - spmv_warp (32768, 1, 1)x(256, 1, 1)	510.05 usecond	826,064	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.62 cycle/nsecond	[16279] spmv_gpu	

GPU Speed Of Light Throughput

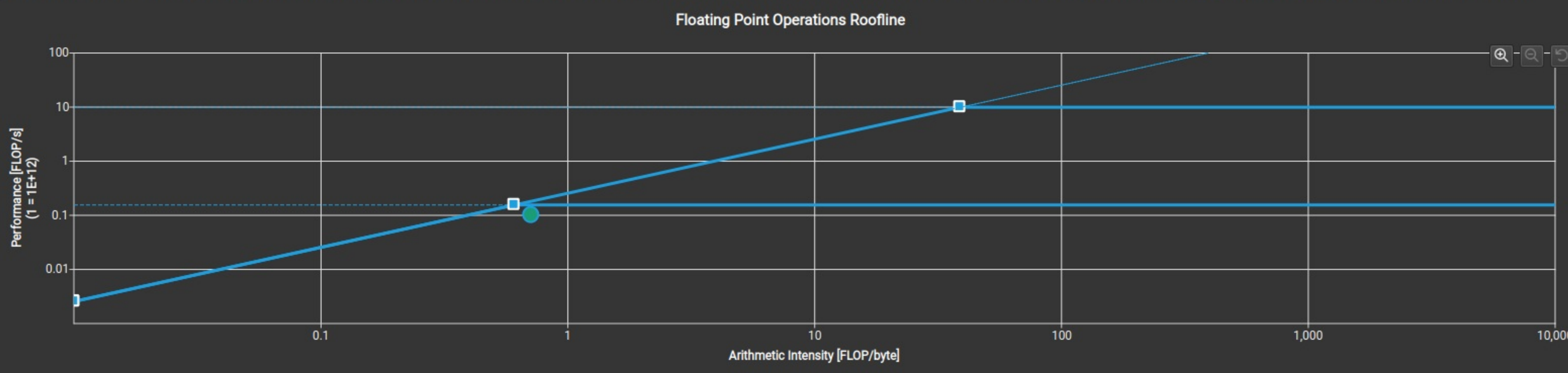
GPU Throughput Rooflines

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	31.75	Duration [usecond]	510.05
Memory Throughput [%]	68.27	Elapsed Cycles [cycle]	826,064
L1/TEX Cache Throughput [%]	88.25	SM Active Cycles [cycle]	822,493.58
L2 Cache Throughput [%]	68.27	SM Frequency [cycle/nsecond]	1.62
DRAM Throughput [%]	54.35	DRAM Frequency [cycle/nsecond]	7.99

High Memory Throughput Memory is more heavily utilized than Compute: Look at the Memory Workload Analysis section to identify the L2 bottleneck. Check memory replay (coalescing) metrics to make sure you're efficiently utilizing the bytes transferred. Also consider whether it is possible to do more work per memory access (kernel fusion) or whether there are values you can (re)compute.

Roofline Analysis The ratio of peak float (fp32) to double (fp64) performance on this device is 64:1. The kernel achieved close to 1% of this device's fp32 peak performance and 0% of its fp64 peak performance. See the Kernel Profiling Guide for more details on roofline analysis.



PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [usecond]	1	# Pass Groups	2
Maximum Buffer Size [Mbytes]	4	Dropped Samples [sample]	0

Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	0.91	SM Busy [%]	22.92
Executed Ipc Active [inst/cycle]	0.92	Issue Slots Busy [%]	22.92
Issued Ipc Active [inst/cycle]	0.92		

Balanced ALU is the highest-utilized pipeline (21.9%) based on active cycles, taking into account the rates of its different instructions. It executes integer and logic operations. It is well-utilized, but should not be a bottleneck.

Memory Workload Analysis

Memory Chart

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/second]	139.00	Mem Busy [%]	68.27
L1/TEX Hit Rate [%]	8.52	Max Bandwidth [%]	66.42
L2 Hit Rate [%]	79.18	Mem Pipes Busy [%]	31.75
L2 Compression Success Rate [%]	0	L2 Compression Ratio	0

L1TEX Global Load Access Pattern
Est. Speedup: 62.50%
The memory access pattern for global loads from L1TEX might not be optimal. On average, only 9.3 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Source Counters section for uncoalesced global loads.

L1TEX Global Store Access Pattern
Est. Speedup: 77.22%
The memory access pattern for global stores to L1TEX might not be optimal. On average, only 4.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Source Counters section for uncoalesced global stores.

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	10.85	No Eligible [%]	76.94
Eligible Warps Per Scheduler [warp]	0.38	One or More Eligible [%]	23.06
Issued Warp Per Scheduler	0.23		

Issue Slot Utilization
Est. Local Speedup: 31.73%
Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 4.3 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this kernel allocates an average of 10.85 active warps per scheduler, but only an average of 0.38 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the Warp State Statistics and Source Counters sections can help, too.

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	47.06	Avg. Active Threads Per Warp	29.75
Warp Cycles Per Executed Instruction [cycle]	47.07	Avg. Not Predicated Off Threads Per Warp	27.88

Long Scoreboard Stalls
Est. Speedup: 31.73%
On average, each warp of this kernel spends 28.4 cycles being stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, texture) operation. Find the instruction producing the data being waited upon to identify the culprit. To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture, attempt to increase cache hit rates by increasing data locality (coalescing), or by changing the cache configuration. Consider moving frequently used data to shared memory. This stall type represents about 60.3% of the total average of 47.1 cycles between issuing two instructions.

Warp Stall Check the Warp Stall Sampling (All Samples) table for the top stall locations in your source based on sampling data. The Kernel Profiling Guide provides more details on each stall reason.

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	18,087,936	Avg. Executed Instructions Per Scheduler [inst]	188,416
Issued Instructions [inst]	18,094,464	Avg. Issued Instructions Per Scheduler [inst]	188,484

FP32 Non-Fused Instructions
Est. Speedup: 2.07%
This kernel executes 262144 fused and 1310720 non-fused FP32 instructions. By converting pairs of non-fused instructions to their fused, higher-throughput equivalent, the achieved FP32 performance could be increased by up to 42% (relative to its current performance). Check the Source page to identify where this kernel executes FP32 instructions.

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	32,768	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	40	Static Shared Memory Per Block [byte/block]	0
Block Size	256	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	8,388,608	Driver Shared Memory Per Block [Kbyte/block]	1.02
Waves Per SM	227.56	Shared Memory Configuration Size [Kbyte]	16.38
Uses Green Context	0	# SMs [SM]	24

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	6
Theoretical Active Warps per SM [warp]	48	Block Limit Shared Mem [block]	16
Achieved Occupancy [%]	90.12	Block Limit Warps [block]	6
Achieved Active Warps Per SM [warp]	43.26	Block Limit SM [block]	24

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	822,493.58	Average L1 Active Cycles [cycle]	822,493.58
Average L2 Active Cycles [cycle]	798,406.31	Average SMSP Active Cycles [cycle]	817,407.58
Average DRAM Active Cycles [cycle]	2,215,536	Total SM Elapsed Cycles [cycle]	19,813,608
Total L1 Elapsed Cycles [cycle]	19,813,608	Total L2 Elapsed Cycles [cycle]	12,902,960
Total SMSP Elapsed Cycles [cycle]	79,254,432	Total DRAM Elapsed Cycles [cycle]	16,306,176

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	2,359,296	Branch Efficiency [%]	100
Branch Instructions Ratio [%]	0.13	Avg. Divergent Branches	0

Uncoalesced Global Accesses
Est. Speedup: 64.46%
This kernel has uncoalesced global accesses resulting in a total of 7336002 excessive sectors (65% of the total 11268162 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The CUDA Programming Guide has additional information on reducing uncoalesced device memory accesses.

L2 Theoretical Sectors Global Excessive

Location	Value	Value (%)
0x7903af282c80 in spmv_warp	7,336,002	100
0x7903af283910 in spmv_warp	0	0
0x7903af2837d0 in spmv_warp	0	0
0x7903af2837c0 in spmv_warp	0	0
0x7903af2837a0 in spmv_warp	0	0

Follow the *rules outputs* to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel.
You could also disable individual sections to focus on selected performance aspects and make profiling faster.